# Roots

## Software Requirements Specification (SRS) Template v1.0

This document is an annotated outline intended for specifying software requirements and is adapted from IEEE 29148-2018.

[Version # 8]
Prepared By: [Team A]
Prepared For: [CSC380 & State University of New York at Oswego]
[Date: 11/09/2026 ]

# Table of Contents

**Table of Contents** **2**
**Change Log** **2**
**1. Introduction** **2**
1.1 Purpose 3
1.2 Scope 4
1.3 Product Overview 5
1.3.1 Product Perspective 5
1.3.1.1 System Interfaces 5
1.3.1.2 User Interfaces 6
1.3.1.3 Hardware Interfaces 6
1.3.1.4 Software Interfaces 6
1.3.1.5 Communication Interfaces 6
1.3.1.6 Memory Constraints 7
1.3.1.7 Site Adaptation 7
1.3.1.8 Interfaces with Services 7
1.3.2 Product Functions 7
1.3.3 User characteristics 8
1.3.4 Limitations 8
1.4 Definitions 8
**2. References** **9**
**3. Requirements** **9**
3.1 Functions 9
3.2 Performance Requirements 11
3.3 Usability Requirements 12
3.4 Interface Requirements 12
3.5 Logical Database Requirements 12
3.6 Design Constraints 13
3.7 Software System Attributes 13
3.8 Supporting Information 13
3.8.1 UML Class Diagram 13
The following UML class diagram provides a structural representation of the Roots system. It illustrates the primary classes, including User, Owner, Contributor, FamilyTree, FamilyMember, Relationship, SuggestRequest, Document, and the ChangeType enumeration, along with their attributes, operations, and associations. The diagram is intended to provide additional context for understanding the organization and interactions of the major components described in the requirements. 13
Figure 3.8.1 UML Diagram for Roots. 14

3.8.2 Use Case Diagram 14
Figure 3.8.2 Use Case Diagram for Roots. 14
3.8.3 Activity Diagram 14
Figure 3.8.3 Activity Diagram for Roots. 14
4. Verification 14
4.1 Functions 14
4.2 Performance Requirements 17
4.3 Usability Requirements 17
4.4 Interface Requirements 18
4.5 Logical Database Requirements 18
4.6 Design Constraints 18
4.7 Software System Attributes 18
4.8 Supporting Information 18
5. Appendix A – Tailoring Policies 19
5.1 Assumptions and dependencies 19
5.2 Acronyms and Abbreviations 19
5.3 Tailoring Policies 20
6. Appendix B – Copyright 21

# Change Log

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| | | | |
| | | | |
| Team A | Sept 18, 2026 | Added UML class diagram to Section 3.8 Supporting Information to illustrate the primary system classes and their relationship | 3 |
| | | | |
| Team A | Oct 2, 2026 | Updated UML, Updated Activity Diagram | 5 |

| | | | |
|---|---|---|---|
| Team A | Oct 16, 2026 | Filled Sections 3 to 5 (requirements, verification, assumptions, acronyms), added definitions, aligned terminology (Roots, Tree Owner, Collaborator), updated interfaces for Django and Google OAuth with link-based invitations. | 6 |
| Team A | Oct 19, 2026 | Narrowed search to members of trees the user owns or collaborates on (removed worldwide relative discovery). Added marriage date and Parent/Child storage rule (FR-5, IR-3), reusable invitation links (FR-10), name search (FR-15), same-tree rule (DB-2), TreeCollaborator (DB-1), PostgreSQL (DC-3), owner-only approval (SA-1), hashed link tokens (SA-2). Clarified that uploaded files are stored on the application server. | 7 |
| Team A | Nov 9, 2026 | Added the ADD_RELATIONSHIP suggestion type so Collaborators can propose relationships (FR-12, IR-6, Section 1.3.3), with the same checks as FR-5 and FR-17 applied at submission and again at approval. Extended the FR-12 to FR-14 verification with an approval-time revalidation test. Documented the 7-day invitation link expiry (FR-10). | 8 |

# 1. Introduction

**Roots** will enable users who upload their extended family data to create a symbolic tree that visually shows family connections. Tree Owners can grow a tree by inviting relatives through shareable invitation links, and a search feature helps users quickly find people within their trees.

## 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for the **Roots system**, a collaborative application designed to help users create, visualize, maintain, and explore family relationships digitally. The system is intended to provide an interactive alternative to traditional paper-based family trees while allowing family information to be preserved and expanded over time.

The vision of the Roots system is to create a long-term platform where users can not only document their family lineage, but also invite relatives to view and contribute to a shared family tree. The system will support visual exploration of family trees, collaboration among family members, invitations, and the storage of important genealogical information and supporting records. It will also consider different family structures and cultural relationships when representing connections.

This SRS will serve as a shared reference for the **stakeholders, clients, development team, testers, and course instructors**, establishing a common understanding of what the system is expected to accomplish and guiding its design, development, and verification.

## 1.2 Scope

The **Roots system** will provide users with a digital and collaborative way to create, view, maintain, and explore family trees. The system is intended to preserve family history, represent different types of family relationships, and help users quickly find people within their family trees.

The scope of the system is:

- Allow users to create, view, and update a visual family tree.
- Represent family relationships such as parents, children, marriages, adoptions, and same-sex marriages.

- Store family information such as names, birth dates, death dates, marriage dates, photographs, and any other genealogical records that users are willing to upload.
- Allow users to zoom into and explore specific portions of a family tree.
- Support collaboration between family members when building or expanding a family tree.
- Allow Tree Owners to invite other people to join a family tree through a shareable invitation link.
- Allow users to search for people by name within the family trees they own or collaborate on.
- Represent family members who do not share a surname, such as through marriage or cultural naming practices.
- Provide standardized symbols and representations for different family relationships and personal information**.**
- Providing a system that can continue to grow as more family members and generations are added.

**We are not building a system that:**

- Proves that two users are biologically related through DNA testing.
- Provides a public directory or search for people outside the trees a user belongs to.

## 1.3 Product Overview

### 1.3.1 Product Perspective

**Roots** is intended to support long-term use, allowing family trees to grow as new generations, relationships, records, and users are added.

The system will interact with external users through account-based access and invitations. Users may contribute family information, photographs, and supporting documents such as birth or adoption records. The Roots system is not currently defined as a component of a larger existing software system; instead, it is being developed as an independent product that may later integrate with additional services or platforms if required.

#### 1.3.1.1 System Interfaces

**Roots** will operate as a standalone web-based application and will not require integration with an existing genealogy or family-tree system for its core functionality. Users will access the system through registered accounts using a supported web browser.

The system will consist of a user-facing web interface, backend application services, and persistent data storage. These components will communicate to process user requests, manage accounts and permissions, retrieve and update family-tree information, and maintain family-related records.

The system may interact with external services when required for supporting functionality asuch as user authentication; uploaded photos and documents are stored on the application server. External service integrations are listed in Section 1.3.1.8.

#### 1.3.1.2 User Interfaces

The user interface will be developed in **JavaScript using the React** framework to provide an interactive and responsive front end for **Roots**. The interface will allow users to easily view and navigate family trees, add or edit family members, manage relationships, upload relevant information, and interact with other features of the system. The interface will focus on being organized, user-friendly, and easy to navigate for people who have no technical knowledge.

#### 1.3.1.3 Hardware Interfaces

**Roots** will be web-based and designed to run on standard consumer hardware such as Desktops, Laptops, Smartphones, and Tablets. The system will support different displays and allow users to navigate the interactive tree using a mouse, trackpad, and touchscreen. No specialized hardware will be required to access the system.

#### 1.3.1.4 Software Interfaces

**Roots** will interact with several software components to provide its functionality. **The web application** will communicate with a Python (Django) backend responsible for processing user requests, managing accounts, and handling family tree information. **The backend will** communicate with a database used to store user profiles, family relationships, names, birth dates, pictures, and other family-related information.

The application may also **interface with external services or APIs** for user authentication (Google OAuth). Uploaded photos and documents are stored on the application server. Any additional third-party services will be listed in Section 1.3.1.8 as they are selected.

#### 1.3.1.5 Communication Interfaces

Communication between the client and server will occur using standard web protocols such as **HTTP/HTTPS**. HTTPS will be used to securely transmit user account information, family tree data, images, and other personal information.

The frontend and backend of the application will communicate through web-based requests and responses. The system may also communicate with external services for account authentication (Google OAuth).

#### 1.3.1.6 Memory Constraints

**Roots** will not have any specific memory requirements beyond what is needed in a standard web browser and device.

#### 1.3.1.7 Site Adaptation

**Roots** is designed as a web-based system and will require minimal site adaptation. Users will access the application through any supported web browser and will not require specialized hardware or software. **Users will only need a supported web browser and an internet connection to access the application.** The interface will be designed to adapt to different screen sizes, including desktop, tablet, and mobile devices.

#### 1.3.1.8 Interfaces with Services

Roots will use Google OAuth 2.0, through the django-allauth library, so users can register and log in with a Google account. Invitations are shared as system-generated links, so no email service is required. No other external services are required for core functionality.

### 1.3.2 Product Functions

**Roots** will provide the major functions needed to create, maintain, explore, and collaboratively expand a digital family tree. Based on the stakeholder meeting notes, the primary product functions are:

- **User Account Management:** Users will be able to register and log in using a Google account.

- **Family Tree Creation:** Users will be able to create and build a digital family tree containing multiple generations and relationships.
- **Family Tree Visualization:** The system will visually display family relationships using standardized symbols and relationship representations.
- **Family Member Management:** Users will be able to add or edit information such as names, birth dates, death dates, marriage dates, photographs, and related family information.
- **Relationship Representation:** The system will support different family relationships, including marriage, adoption, parent-child relationships, and same-sex marriages.
- **Document and Record Support:** Users will be able to associate supporting records, such as birth certificates and adoption documents, with family members.
- **Interactive Tree Navigation:** Users will be able to zoom in and explore specific sections or branches of a family tree.
- **Collaboration:** Multiple users will be able to contribute to and expand a shared family tree.
- **Invitation System:** Tree Owners will be able to generate a shareable invitation link that lets other users join a family tree as Collaborators.
- **Member Search:** Users will be able to search for family members by name within trees they own or collaborate on.
- **Data Privacy:** Tree data will be visible only to the Tree Owner and invited Collaborators.
- **Family History Preservation:** The system will maintain family information over time so that the family tree can continue to grow across generations.

### 1.3.3 User characteristics

**Roots** has two user roles. **Tree Owners** create family trees, add and delete members and relationships, approve or reject suggested changes, and generate invitation links. **Collaborators** join a tree through an invitation link, view the tree, and submit suggested additions, edits, relationships, and historical media for the Tree Owner to approve. Collaborators see a simplified, guided view without management controls. All users are assumed to have basic web-browsing skills but no technical or genealogical expertise.

### 1.3.4 Limitations

**Data Verification: Roots** will rely on user-submitted information, so it will not independently verify the accuracy of the data such as birthdates, birth locations, and familial relationships. Due to this, the integrity of the family tree is based upon the accuracy of the information provided by the users.

**Data Completeness:** Unlike platforms that aggregate historical records through web scraping or third-party archives, our system is limited to users' knowledge and input. As a result, the family tree may have gaps where information is unknown.

## 1.4 Definitions

Terms specific to Roots are defined below. Abbreviations are listed in Appendix 5.2.

| Term | Definition |
|---|---|
| *Example: context of use* | *users, tasks, equipment (hardware, software and materials), and the physical and social environments in which a product is used* |
| System Under Development | The system that is actively being developed. |
| Tree Owner | The user who creates a family tree. Has full rights to edit it, approve or reject suggestions, and generate invitation links. |
| Collaborator | A user who joined a tree through an invitation link. Can view the tree and submit suggestions, but cannot change it directly. Shown as Contributor in the UML class diagram. |
| Family Member | A person represented as a node in a family tree. Not necessarily a user of the system. |
| Relationship | A link between two family members: Parent, Child, Spouse, Sibling, or Adoptive Parent. |
| Suggestion | A change proposed by a Collaborator that is applied only if the Tree Owner approves it. Stored as a SuggestRequest. |
| Invitation Link | A unique, revocable URL generated by a Tree Owner that gives Collaborator access to one tree. |

# 2. References

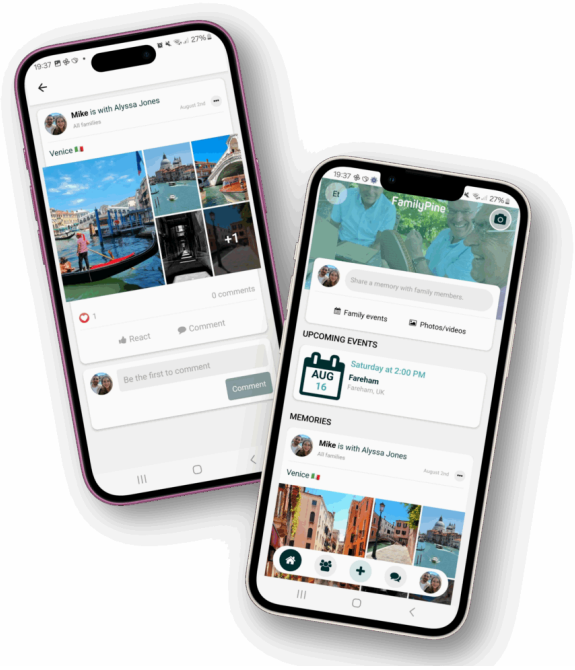



**Figure 1. Example Landing Page. Source: [familypine.com](familypine.com)**

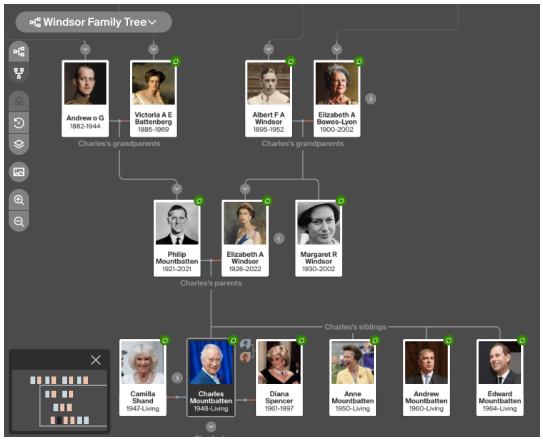



**Figure 2. Example Family Tree. Source: [ancestry.com](ancestry.com)**

# 3. Requirements

Each requirement below has a unique ID by type: FR (functional), PR (performance), UR (usability), IR (interface), DB (database), DC (design constraint), and SA (system attribute). Section 4 states how each requirement will be verified.

## 3.1 Functions

**Accounts**

FR-1: The system shall allow a user to register and log in using a Google account.

FR-2: The system shall allow a logged-in user to log out, ending their session.

**Tree management (Tree Owner)**

FR-3: The system shall allow a logged-in user to create a family tree with a name and an optional description; that user becomes the tree's Tree Owner.

FR-4: The system shall allow a Tree Owner to add, edit, and delete family members, each with first name, last name, gender, and optional birth date, death date, and photo.

FR-5: The system shall allow a Tree Owner to create a relationship of type Parent, Child, Spouse, Sibling, or Adoptive Parent between two family members of the same tree, including same-sex spouses. A Spouse relationship may include an optional marriage date. A Child relationship is stored as the reverse of a Parent relationship.

FR-6: The system shall allow a Tree Owner to attach documents (PDF, JPG, PNG, up to 10 MB) to a family member.

**Visualization**

FR-7: The system shall display a family tree as a diagram, using a distinct line or symbol for each relationship type.

FR-8: The system shall allow users to zoom and pan the tree using a mouse, trackpad, or touchscreen.

FR-9: The system shall show a member's profile (details and attached documents) when the member is selected.

**Collaboration**

FR-10: The system shall allow a Tree Owner to generate a unique invitation link for a tree, and to revoke it. A link may be used by multiple people until it is revoked or expires (links expire 7 days after creation).

FR-11: When a logged-in user opens a valid invitation link, the system shall add that user to the tree as a Collaborator.

FR-12: The system shall allow a Collaborator to submit a suggestion of type ADD_MEMBER, UPDATE_MEMBER, ADD_MEDIA, or ADD_RELATIONSHIP with a description. An ADD_RELATIONSHIP suggestion shall pass the checks in FR-5 and FR-17 when submitted and again when approved.

FR-13: The system shall show the Tree Owner a list of pending suggestions, and allow the Owner to approve or reject each one.

FR-14: When a suggestion is approved, the system shall apply the change to the tree; when rejected, the system shall discard it without changing the tree.

**Search**

FR-15: The system shall allow users to search family members by first or last name (partial, case-insensitive) and return matching members only from trees the user owns or collaborates on.

**Input validation and errors**

FR-16: The system shall reject a death date earlier than the birth date and display an error message.

FR-17: The system shall reject a relationship that would make a member their own ancestor.

FR-18: The system shall reject uploads that are not PDF, JPG, or PNG, or that exceed 10 MB, and display the reason.

FR-19: If an invitation link is invalid or revoked, the system shall display an error and shall not grant access.

FR-20: If a save fails due to a network or server error, the system shall keep the user's entered data on screen and display a retry message.

FR-21: The system shall store, with each suggestion, the version of the family member it was based on. If that member changed before the suggestion is reviewed, the system shall mark the suggestion as outdated and show the Tree Owner the current and proposed values side by side.

FR-22: If two suggestions are approved at the same time, the system shall apply them one at a time so that neither approval overwrites the other without the Tree Owner seeing it.

## 3.2 Performance Requirements

PR-1: The family tree view shall render within 3 seconds for a tree of up to 200 members.

PR-2: The system shall support at least 50 concurrent users without errors.

PR-3: Search results shall be returned within 2 seconds.

PR-4: Saving an edit or approving a suggestion shall complete within 2 seconds.

## 3.3 Usability Requirements

UR-1: A first-time user shall be able to create a tree and add 3 family members within 10 minutes without outside help.

UR-2: A Collaborator shall be able to submit a suggestion in 3 clicks or fewer from the member's profile.

UR-3: The interface shall be usable on screen widths from 360 px (phone) to 1920 px (desktop).

UR-4: At least 80% of usability-test participants shall rate overall ease of use 4 out of 5 or higher.

UR-5: The system shall ask for confirmation before a Tree Owner deletes a family member or relationship.

## 3.4 Interface Requirements

The interfaces below are described in Section 1.3.1. Each is listed with its purpose, source or destination, format, and limits.

IR-1 Google Login: Authenticates the user. Input: Google OAuth 2.0 token from Google. Output: an authenticated session. Timing: login shall complete within 5 seconds.

IR-2 Family Member Form: Adds or edits a person. Input from the user: first name and last name (text, 1 to 50 characters each), gender, birth date and death date (MM/DD/YYYY, optional). Output: a saved family member shown in the tree.

IR-3 Relationship Form: Links two family members. Input: two member IDs and a type (Parent, Child, Spouse, Sibling, Adoptive Parent), plus an optional marriage date (MM/DD/YYYY) for Spouse. Output: a new line on the tree.

IR-4 Document Upload: Attaches a record to a member. Input: PDF, JPG, or PNG file, 10 MB maximum. Output: the stored file linked to the member's profile.

IR-5 Invitation Link: Grants Collaborator access. Output: a unique URL generated for the Tree Owner. Input: the URL opened by an invited user.

IR-6 Suggestion Form: Proposes a change. Input from a Collaborator: change type (ADD_MEMBER, UPDATE_MEMBER, ADD_MEDIA, ADD_RELATIONSHIP) and description. For ADD_RELATIONSHIP, the input also includes two member IDs, a relationship type, and an optional marriage date, validated as in IR-3. Output: a pending suggestion shown to the Tree Owner.

IR-7 REST API: All frontend-backend communication uses JSON over HTTPS.

## 3.5 Logical Database Requirements

DB-1 Entities: The database shall store User, FamilyTree, FamilyMember, Relationship, SuggestRequest, Document, Invitation, and TreeCollaborator records, as shown in the ER diagram.

DB-2 Relationships: Each Relationship shall reference exactly two existing FamilyMembers in the same tree. Each FamilyTree shall have exactly one Owner. A Relationship's two members, a SuggestRequest's target member, and a TreeCollaborator's invitation shall belong to the same tree as the record that references them.

DB-3 Integrity: The system shall reject a FamilyMember whose death date is before the birth date, and shall prevent a member from being their own ancestor.

DB-4 Access: A user shall only read trees they own or collaborate on. Only the Tree Owner shall write directly to a tree; Collaborators write only SuggestRequest records.

DB-5 Frequency of use: Tree and member data are read on every tree view; writes occur when members are added, edited, or suggestions are approved.

DB-6 Retention: Data shall be kept until the Tree Owner deletes it. Deleting a tree shall delete its members, relationships, suggestions, documents, invitations, and collaborator memberships.

DB-7 Security: Uploaded files shall be stored outside the public web root and served only to authorized users.

## 3.6 Design Constraints

DC-1 Schedule: The system shall be completed and submitted by December 4, 2026.
DC-2 Team: The system is built by a team of 4 students within one semester.
DC-3 Technology: The frontend shall use React, the backend shall use Python with Django, the database shall use PostgreSQL, and authentication shall use Google OAuth 2.0.
DC-4 Privacy: Because trees contain personal data about living people, tree data shall not be visible to users who are not the Tree Owner or an invited Collaborator.
DC-5 Standards: This document follows IEEE 29148-2018 as adapted by the course template.

## 3.7 Software System Attributes

SA-1 Security: All traffic shall use HTTPS. Role-based access shall be enforced on the server, so a Collaborator cannot change tree data directly even by calling the API, and only the Tree Owner can approve or reject suggestions.
SA-2 Security: Invitation links shall be unguessable random tokens, only a hash of each token shall be stored, and a Tree Owner shall be able to revoke a link.
SA-3 Reliability: A failed save shall not leave partial data; each write shall run inside a database transaction.
SA-4 Availability: The deployed system shall be available at least 95% of the time during the demo and grading period.
SA-5 Maintainability: All code shall be stored in the team GitHub repository, and every change shall be merged to main through a reviewed pull request.
SA-6 Portability: The system shall run on current versions of Chrome, Firefox, Safari, and Edge on desktop and mobile.

## 3.8 Supporting Information

### 3.8.1 UML Class Diagram

The following UML class diagram provides a structural representation of the Roots system. It illustrates the primary classes, including User, Owner, Contributor, FamilyTree, FamilyMember, Relationship, SuggestRequest, Document, and the ChangeType enumeration, along with their attributes, operations, and associations. The diagram is intended to provide additional context for understanding the organization and interactions of the major components described in the requirements.

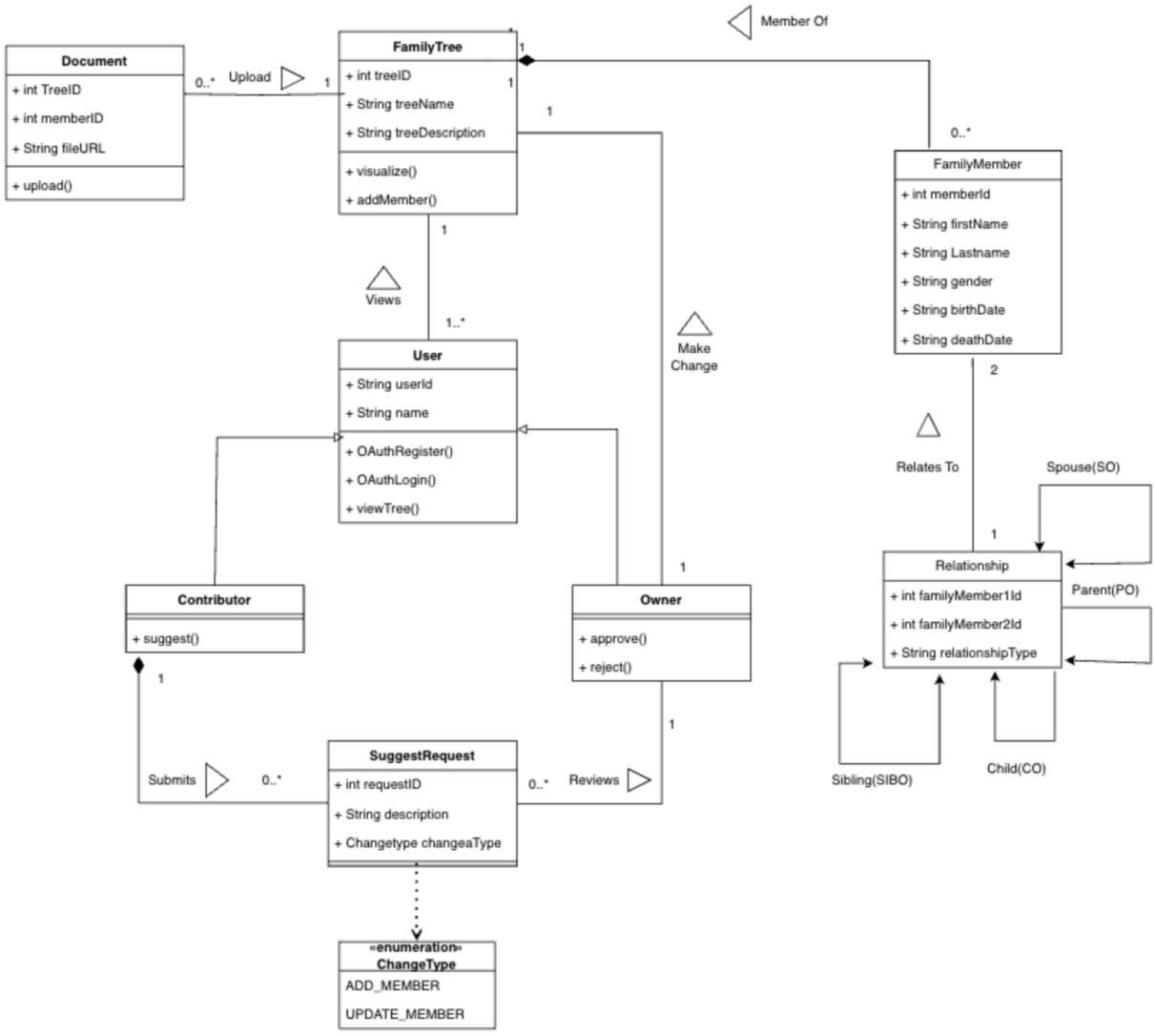



**Figure 3.8.1 UML Diagram for Roots.**

### 3.8.2 Use Case Diagram

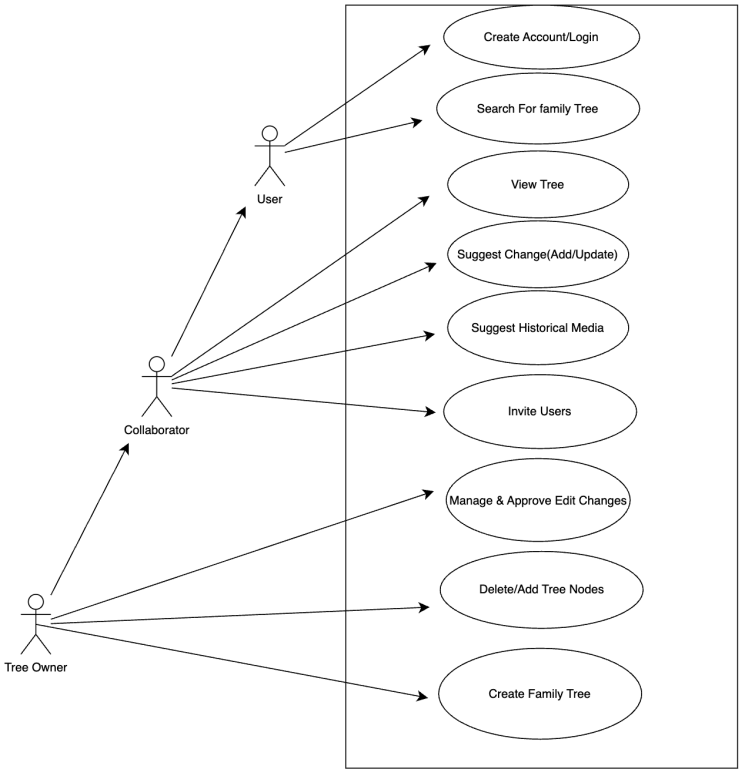



**Figure 3.8.2 Use Case Diagram for Roots.**

### 3.8.3 Activity Diagram

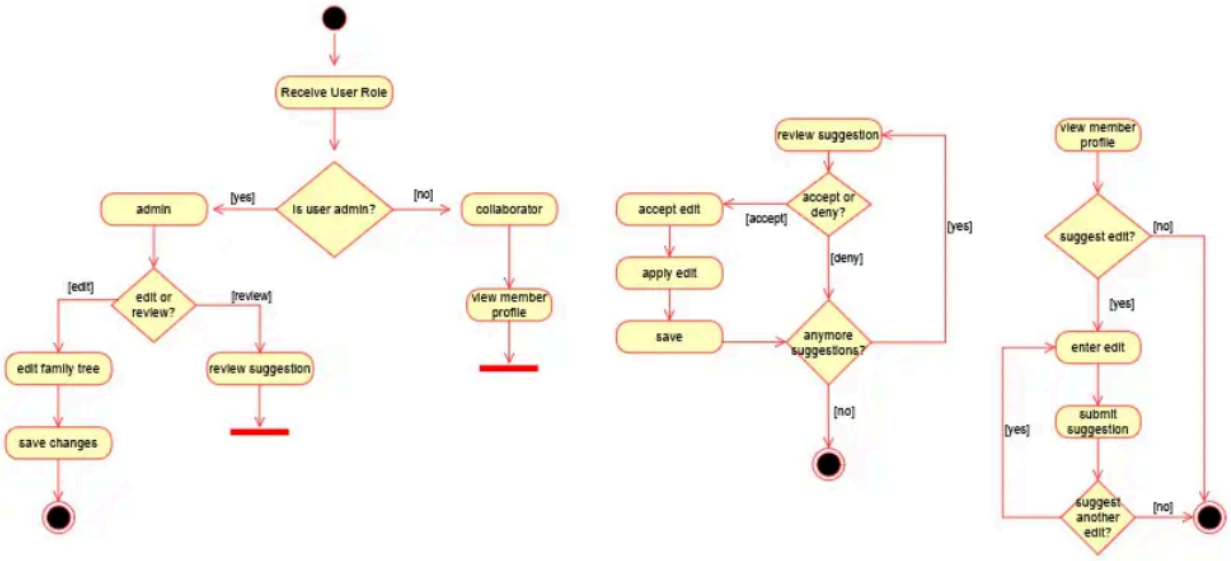



**Figure 3.8.3 Activity Diagram for Roots.**

# 4. Verification

Each requirement in Section 3 is verified by one of four methods: Test (automated unit or integration test), Demonstration (performed live in the running system), Inspection (review of code, configuration, or documents), or Analysis (measurement or calculation). Tests are run by the testing lead during Phase 5 of the project schedule.

## 4.1 Functions

FR-1, FR-2 (Demonstration): Log in with a Google account, confirm the user lands on their dashboard, log out, and confirm protected pages redirect to login.
FR-3 to FR-6 (Test): Integration tests create a tree, add, edit, and delete members, create each relationship type (including same-sex spouses), and attach a PDF; each test asserts the saved data matches the input.
FR-7 to FR-9 (Demonstration): Load a 3-generation sample tree, check each relationship type has its own symbol, zoom and pan with mouse and touchscreen, and open a member profile.
FR-10, FR-11, FR-19 (Test): Generate a link, open it as a second user and assert Collaborator access; revoke it and assert the next attempt is refused with an error.
FR-12 to FR-14 (Test): As a Collaborator submit one suggestion of each type; as the Owner approve one and reject one, and assert only the approved change appears in the tree. For ADD_RELATIONSHIP, also make a pending proposal invalid before review (for example, the

Owner adds the same relationship directly) and assert that approval is refused with an error and no duplicate relationship is created.
FR-15 (Test): Search a known first name and a partial last name in mixed case, and assert only members from accessible trees are returned.
FR-16 to FR-18 (Test): Unit tests submit a death date before the birth date, a circular ancestor link, and a 12 MB or .exe upload, and assert each is rejected with an error message.
FR-20 (Demonstration): Stop the backend, submit a form, and confirm the entered data stays on screen with a retry message.
FR-21, FR-22 (Test): Create two suggestions for the same member, approve the first, and assert the second is marked outdated; send two approvals at once and assert both changes are applied without loss.

## 4.2 Performance Requirements

PR-1, PR-3, PR-4 (Analysis): Load a seeded 200-member tree and time page render, search, and save in browser developer tools; each must meet its limit in 5 of 5 runs.
PR-2 (Test): Run a load test (for example, Locust) with 50 simulated users for 5 minutes and confirm zero failed requests.

## 4.3 Usability Requirements

UR-1, UR-2, UR-4 (Demonstration): Run a usability session with at least 5 participants who have not used Roots; time task completion, count clicks, and collect a 1 to 5 ease-of-use rating.
UR-3 (Inspection): Check every screen at 360 px, 768 px, and 1920 px widths in browser developer tools.
UR-5 (Test): Click delete on a member and assert a confirmation dialog appears and nothing is deleted if cancelled.

## 4.4 Interface Requirements

IR-1 to IR-6 (Test): API tests send valid and invalid input to each endpoint (empty names, names over 50 characters, wrong date format, unsupported relationship type, oversized file) and assert valid input is saved and invalid input returns an error.
IR-7 (Inspection): Review network traffic in browser developer tools to confirm all requests use HTTPS and JSON.

## 4.5 Logical Database Requirements

DB-1, DB-2 (Inspection): Compare the Django models and migrations against the ER diagram.
DB-3, DB-6 (Test): Unit tests assert invalid dates and circular relationships are rejected, and that deleting a tree removes all of its child records.
DB-4, DB-7 (Test): As a user with no access, request another tree’s data and a stored file URL, and assert both return HTTP 403.

## 4.6 Design Constraints

DC-1, DC-2 (Inspection): Compare progress against the project Gantt chart each week; the final submission must be made by December 4, 2026.
DC-3 (Inspection): Review the repository to confirm React, Django, and Google OAuth are used.
DC-4 (Test): Covered by the DB-4 access test.
DC-5 (Inspection): Instructor and team review of this document against the course template.

## 4.7 Software System Attributes

SA-1 (Test): Call write endpoints directly with a Collaborator's session and assert HTTP 403; confirm HTTP requests redirect to HTTPS.
SA-2 (Test): Assert link tokens are at least 32 random characters and that a revoked link is refused.
SA-3 (Test): Force an error midway through a multi-step save and assert no partial records remain.
SA-4 (Analysis): Monitor the deployed site with an uptime checker during the final two weeks.
SA-5 (Inspection): Review GitHub history to confirm every merge to main came through a reviewed pull request.
SA-6 (Demonstration): Run the main user flows in Chrome, Firefox, Safari, and Edge on desktop and on one phone.

## 4.8 Supporting Information

The diagrams in Section 3.8 are verified by Inspection: the team reviews them against the Django models and implements user flows before final submission and updates them if they differ.

# 5. Appendix A – Tailoring Policies

## 5.1 Assumptions and dependencies

A-1: Users have a Google account, since it is the only login method.
A-2: Users access Roots with a current version of Chrome, Firefox, Safari, or Edge and a working internet connection.
A-3: Information entered by users is assumed to be accurate; Roots does not verify it (see Section 1.3.4).
A-4: Google OAuth 2.0 and the django-allauth library remain available and compatible with the Django version used. A change to Google's login API would affect FR-1.
A-5: A hosting platform supporting Python/Django and file storage is available for deployment by the Final Deployment date on the Gantt chart.

## 5.2 Acronyms and Abbreviations

| Term | Definition | Abbreviation | Source (if applicable) |
|---|---|---|---|
| *Example: context of use* | *users, tasks, equipment (hardware, software and materials), and the physical and social environments in which a product is used* | *COU* | *[SOURCE: ISO/IEC 25000:2014, 4.2]* |
| System Under Development | The system that is actively being developed. | SUD | |
| Software Requirements Specification | A document that defines what a system must do and the constraints it must meet. | SRS | ISO/IEC/IEEE 29148-2018 |
| User Interface | The screens and controls a user interacts with. | UI | |
| Application Programming Interface | A defined set of requests one program uses to talk to another; here, between the frontend and backend. | API | |
| Hypertext Transfer Protocol Secure | The encrypted protocol used for all traffic between the browser and server. | HTTPS | |
| Open Authorization | A standard that lets users log in with an existing account (Google) without sharing their password with Roots. | OAuth | |
| Create, Read, Update, Delete | The four basic operations on stored data. | CRUD | |

| | | | |
|---|---|---|---|
| Role-Based Access Control | Granting permissions by role (Tree Owner or Collaborator) rather than per user. | RBAC | |
| Entity-Relationship | A diagram showing database tables and how they relate. | ER | |
| Unified Modeling Language | A standard notation for software diagrams such as class, use case, and activity diagrams. | UML | |
| JavaScript Object Notation | The text format used for data sent between the frontend and backend. | JSON | |

## 5.3 Tailoring Policies

Tailoring is not a requirement to bring the document into compliance with the standards set by IEEE 29148-2018. Tailoring should only occur when conformance to the standard is not possible or practical. The act of tailoring is the modification and/or removal of one of the content sections outlined in this document; adding additional information items for organization is not considered tailoring. Tailoring should only occur when factors or circumstances:

- surround an organization that is using the document
- influence a project using this document to meet an agreement
- reflect the needs of an organization.

When tailoring the document, the following activities shall be implemented:

- Identify and document the circumstances that may influence tailoring.
    - novelty, size and complexity
    - stability of operating environments
    - variety in operating environments
    - starting date and duration
    - emerging technology
    - availability of services of enabling systems
    - other standards with which the document needs to conform.
- Identify and get input from all parties impacted by the tailoring process.
    - Such as stakeholders, contributors, and other interested parties
- Delete the information content that requires tailoring.

# 6. Appendix B – Copyright

This document is based on a template meeting the ISO/IEC/IEEE 29148-2018 standard, available at https://www.iso.org/standard/72089.html. Template authors are:

**Dr. rer. nat. Bastian Tenbergen,**
Associate Professor of Software Engineering
bastian@tenbergen.org

**Mikayla Conner-Spagnola**, MA
Independent Consultant
mconner@oswego.edu

Department of Computer Science
State University of New York at Oswego
Oswego, NY 13126, United States

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License. For more information, please see http://creativecommons.org/licenses/by-sa/4.0/